CORVINUS UNIVERSITY of BUDAPEST

Machine Learning in Practice I
Lecture 10

Antal Jakovac, 2025

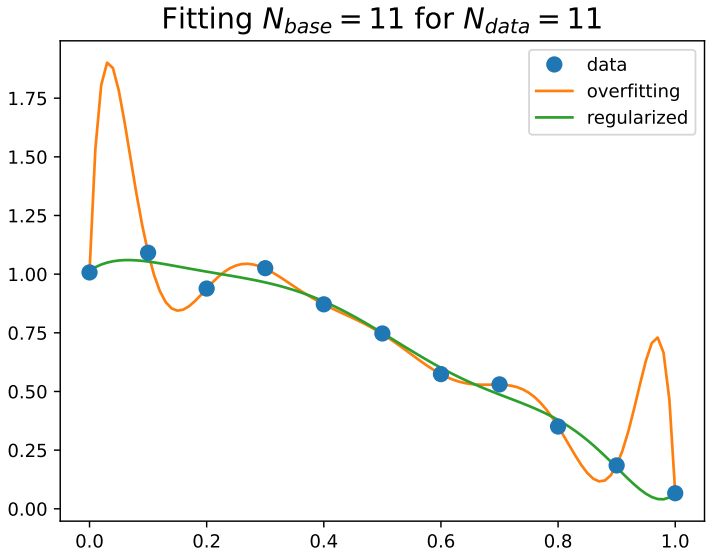
Linear regression
inter- and extrapolation

Linear regression

- interpolation vs. extrapolation
- after function fitting we can predict values
- data lay in $t_n \in T_{range} = [t_{min}, t_{max}]$
  - **interpolation**: prediction for $t \in T_{range}$
  - **extrapolation**: prediction for $t \notin T_{range}$
- dangers of interpolation → overfitting
- dangers of extrapolation → …

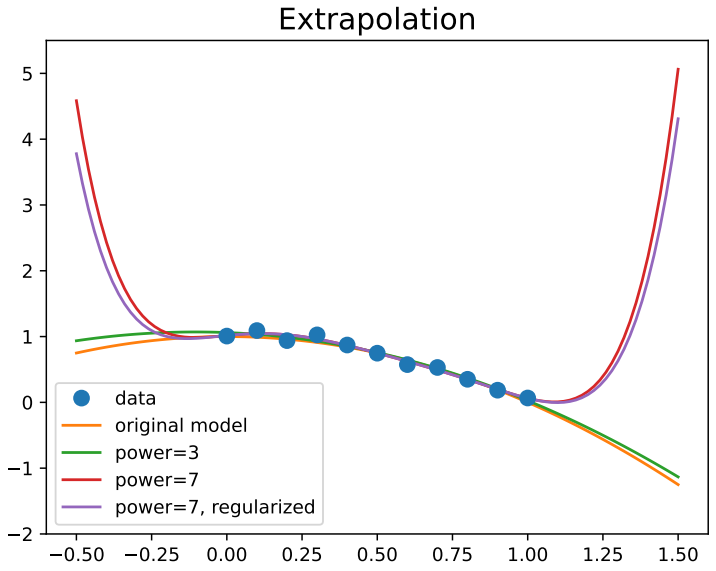
Linear regression

- dangers of interpolation → **overfitting**
- example: fit polynomial basis to data points
- original model $y = 1 - x^2 + \xi$ with $N_{data} = 11$
- fitted model: $N_{basis} = 11$
  with and or without regularization
- result:
  - exact fit, hits all points → bad interpolation
  - worse at the edges
  - regularization helps



Linear regression

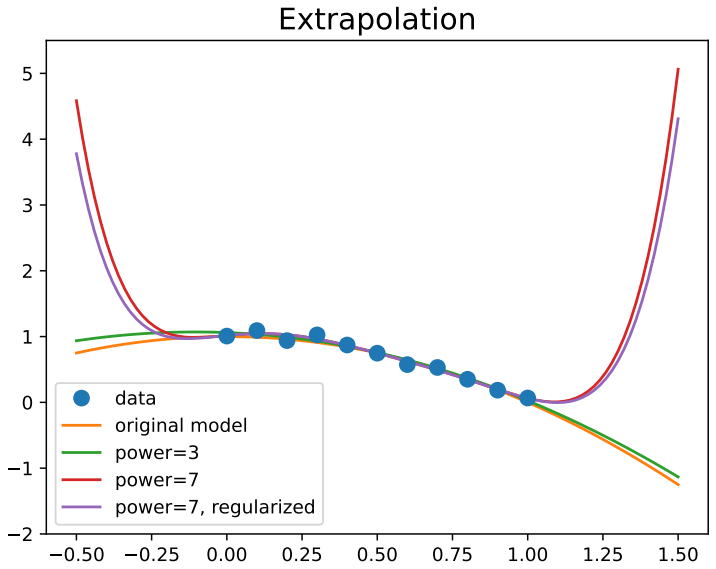
- dangers of extrapolation → **avoid extrapolation**
- fitting lower and higher order polynomials
- results:
  - inside the data range all fits are correct
  - outside all fail
  - the more robust (fewer parameters) the better
  - regularization does not help!



Linear regression

- dangers of extrapolation → **avoid extrapolation**
- fitting lower and higher order polynomials
- results:
  - inside the data range all fits are correct
  - outside all fail
  - the more robust (fewer parameters) the better
  - regularization does not help!

☠ proceed and perish! ⚠



Linear regression

- controlling extrapolation
  - use bounded basis → avoid polynomials!
  - require asymptotic behaviour (i.e. function form at infinity)
    → transforms extrapolation to interpolation
  - use the model requiring the fewest parameters (robust modelling)
  - extend data in a sensible way

Nonlinear regression

Nonlinear regression

- we can have a general data model $y = f(x;\omega)$
- minimize $\chi^2(\omega) = \sum_{n=1}^{N} (y_n - f(x_n;\omega))^T C_n^{-1} (y_n - f(x_n;\omega))$
- no direct methods → recursive approach
  - steepest descent method
  - random algorithms → Metropolis method with simulated annealing
  - challenges at higher dimensions

Steepest descent method

- iteration
  - choose a starting point ω_0
  - establish an iteration $\omega_n \Rightarrow \omega_{n+1}$
  - try to ensure that $\chi^2(\omega_{n+1}) < \chi^2(\omega_n)$
- in steepest descent method: linear approximation around ω_n

$$\chi^2(\omega_n + d\omega) - \chi^2(\omega_n) = d\omega \cdot \nabla \chi^2(\omega_n) + \ldots$$

- length of a vector $|v|^2 = v \cdot v > 0$ $\rightarrow$ choose $d\omega = -r \nabla \chi^2(\omega_n)$
- in linear approximation $\chi^2(\omega_n + d\omega) - \chi^2(\omega_n) = -r \left| \nabla \chi^2(\omega_n) \right|^2 + \ldots < 0$

Steepest descent method

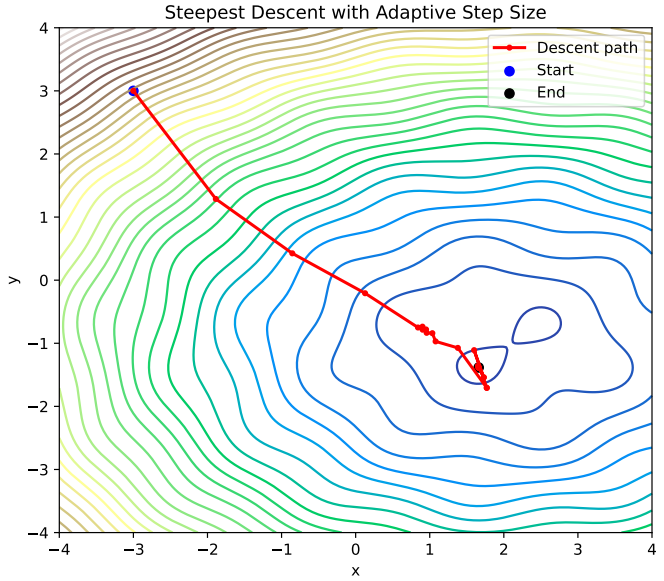
- advantages:
  - simple, low cost
  - requires only first derivative → numerically or automatic differentiation
  - flexible: stochastic and mini batch versions in high dimensions
- disadvantages:
  - slow (linear convergence), especially near minimum
  - for ill-conditioned functions long iteration path → conjugate gradient
  - sensitive to local minima → use momentum (Nesterov acceleration)
  - can lead to runaway solutions → adaptive step size, gradient clipping

Adaptive step length

- in the iteration change the step length r
  - if $\chi^2(\omega_n - r|\nabla\chi^2|) < \chi^2(\omega_n)$ → choose $r \Rightarrow \alpha_{up} r$, $\alpha_{up} > 1$
  - else choose $r \Rightarrow \alpha_{down} r$, $\alpha_{down} < 1$
  - if $r < r_{min}$ or $n_{step} > N_{iter}$ stop iteration
- 2D example with

$$f(x,y) = (x-2)^2 + (y+1)^2 + \sin 3x \sin 3y$$

$$\alpha_{up} = 1.1,\ \alpha_{down} = 0.5,\ N_{iter} = 200$$



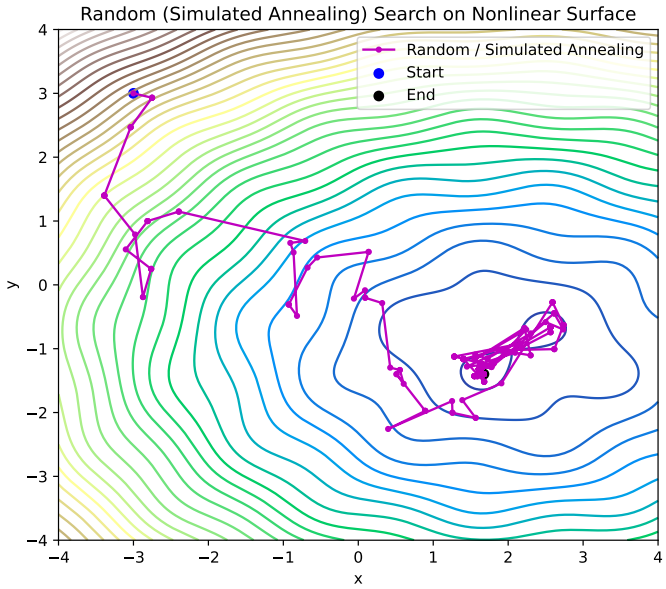
Simulated annealing

- choose a random direction n and a positive number α
- if $\chi^2(\omega_n+\alpha n)<\chi^2(\omega_n)$ → choose $\omega_{n+1}=\omega_n+\alpha n$
- else still accept $\omega_{n+1}=\omega_n+\alpha n$ with a probability $P\sim e^{-\Delta\chi^2/T}$
- T is the “temperature” controlling exploration
- the values in $\{\omega_0,\omega_1,\ldots,\omega_n,\ldots\}$ have a distribution $p(\omega)\sim e^{-\chi^2(\omega)/T}$
  - can be used in Monte Carlo simulation (Metropolis algorithm)
- if we need the minimum, decrease T → simulated annealing (SA)

$$T \Rightarrow \alpha_{cooling} T$$

Simulated annealing

- the same example as before $\alpha_{cooling} = 0.98$
- advantages:
  - no gradient is needed
  - can escape local minima
  - parallelizable
- drawbacks:
  - slow convergence
  - difficult to use in batch methods → rarely used in DNNs



Feature optimization
Perceptron

Linear Features

- we can tailor features instead of the original ones
- simplest: linear combination → looks for a hyperplane to separate classes
- can be generalized → curved separator surface
- implementation:
  - perceptron (linear)
  - Support Vector Machine (linear + nonlinear)
  - Extreme Learning Machine (ELM) (nonlinear)

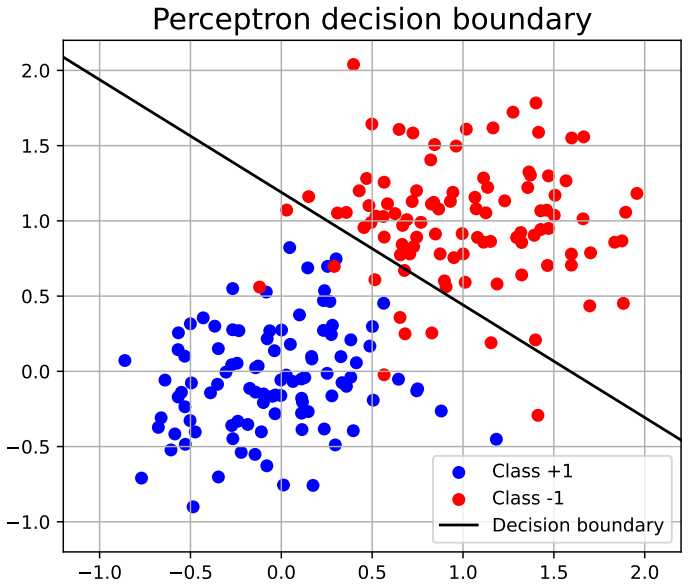
Perceptron

- one of the first AI algorithm (Rosenblatt, 1958)
  → newspapers: *"the electronic brain that can walk, talk, see, write, reproduce itself, and be conscious of its existence"*
- idea:
  - two classes with labels $y = \pm 1$, data are d dimensional $x \in \mathbb{R}^d$
  - look for a vector w and a constant b that satisfies $y = \mathrm{sign}\left(x \cdot w + b\right)$
  - possible loss function to be maximized $L = \sum_{n=1}^{N} y_n \left(x_n \cdot w + b\right)$
  - Rosenblatt's algorithm: if we find a misclassified point x_n

$$w \Rightarrow w + \eta\, y_n x_n, \quad b \Rightarrow b + \eta\, y_n, \quad \eta > 0$$

Perceptron

- example: 2D points
- accuracy: 95.5%



Perceptron

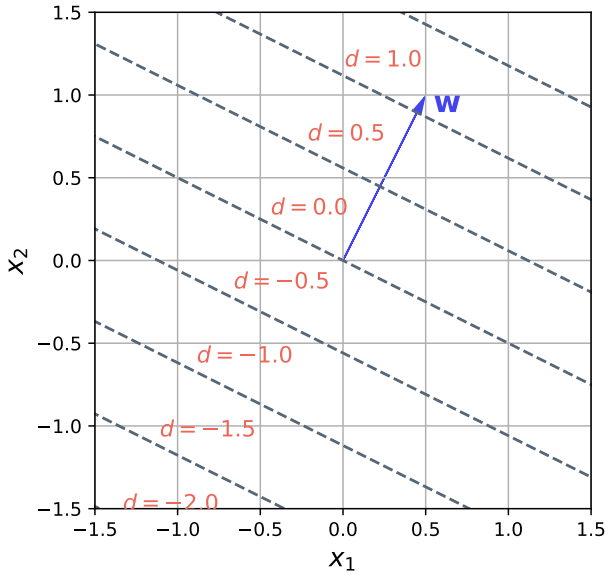
classification by analytic expression → several classes

- class labels → analytic expressions
- one-hot encoding:
  - $i \Rightarrow (0,0,\ldots,1,\ldots,0) = e_i$, the *i*th unit vector
  - n classes → n dimensional output
- other (vector) encodings:
  - $i \Rightarrow v_i \in \mathbb{R}^D$, D dimensional output
  - mapping is optimized by some condition (e.g. semantics for words)

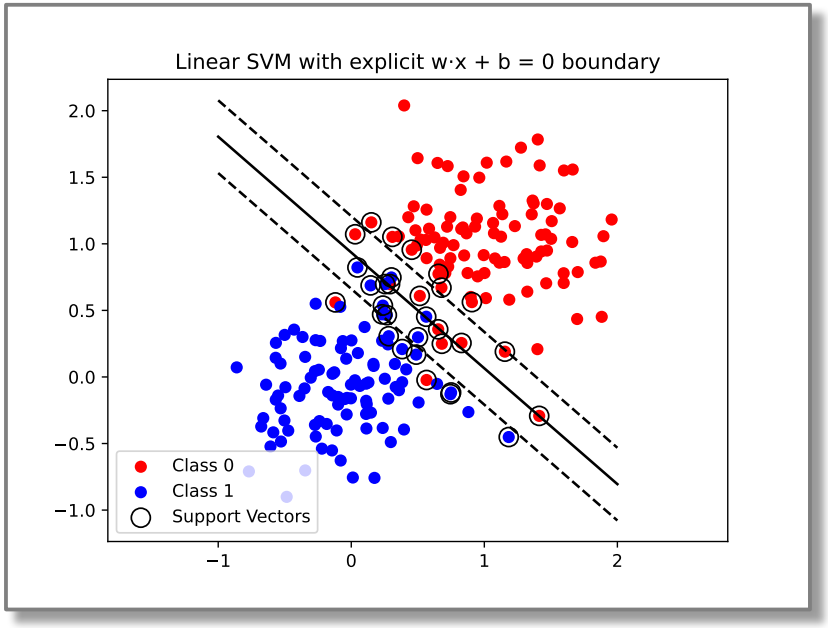
Support Vector Machine

Support Vector Machine

- improved perceptron to maximize margin
- logic:
  - define a *w* vector → the selective coordinate is $x \cdot w$
  - try to separate the two sets using this feature
  - for optimal w use 1D methods (e.g. hinge loss)
  - in the training use the nearest point to write up w → support vectors
  - Python package: svm



Support Vector Machine

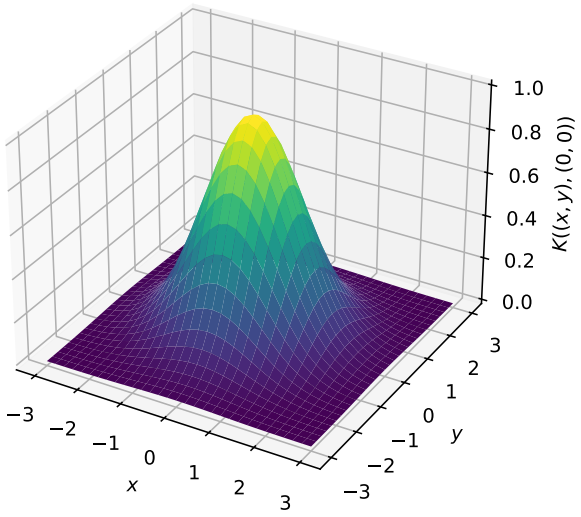



```
# Support Vector Machine example
from sklearn import svm
from sklearn.metrics import accuracy_score

clf = svm.SVC(kernel='linear', C=1.0)
clf.fit(X, y)

# 🔮 Predict on the test set
y_pred = clf.predict(X_test)

# 🧮 Evaluate
acc = accuracy_score(y_test, y_pred)

→ acc = 96.5%
```

Support Vector Machine

- we can use also nonlinear combinations (kernel)
- most used: Gaussian kernel
  - around each point we take a “bump” $x \to e^{-\gamma(x-x_0)^2}$
  - complete kernel is a linear combination (using ±1 as labels)

$$x \to \sum_n \alpha_n y_n e^{-\gamma(x-x_n)^2} + b$$



  - resulting surface is a landscape with hills and valleys
  - in practice it is enough to consider only a small number of points (support vectors) to represent the surface

Support Vector Machine

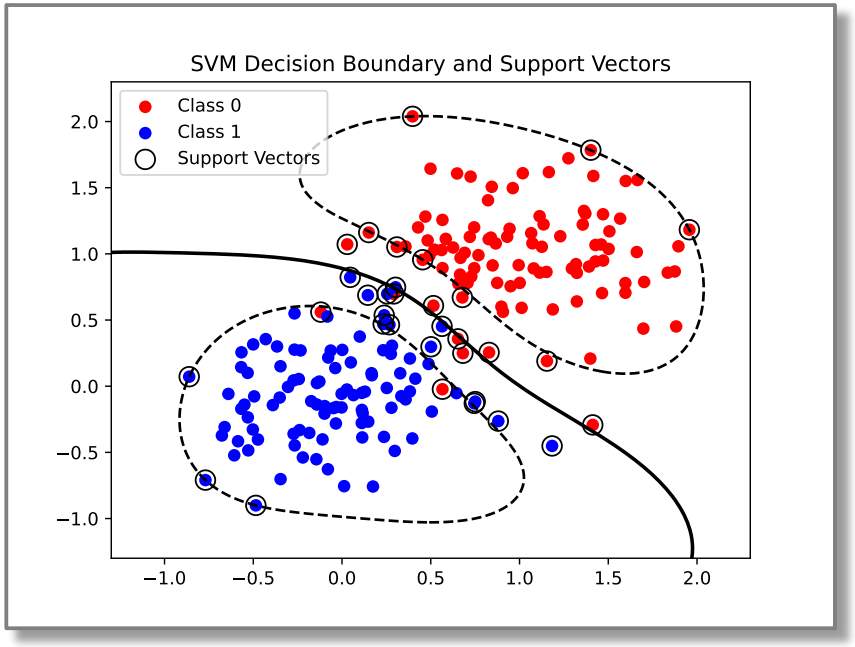
- we can use arbitrary kernel $x \to \sum_n \alpha_n y_n K(x_i, x; \Theta) + b$
- fitting and learning:
  - linear combination parameters α_n, b are learned using data
  - kernel parameters Θ are fixed in the learning process, and tuned for best performance (using grid search, cross validation)
  - Deep Neural Networks work in a similar manner, there the functional space is a nested (layered) function composition.

$$x \to W_n \sigma(W_{n-1} \sigma(\ldots \sigma(W_1(x))\ldots))$$

  - decision based on a single (although) complicated feature

Support Vector Machine

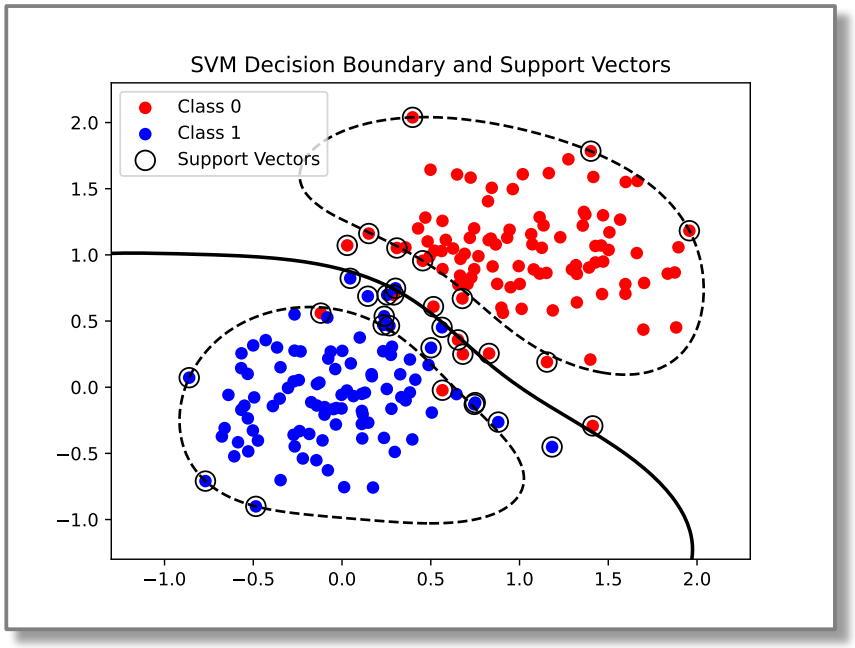
- in the code we should use the *'rbf'* kernel *(Radial Basis Function)*

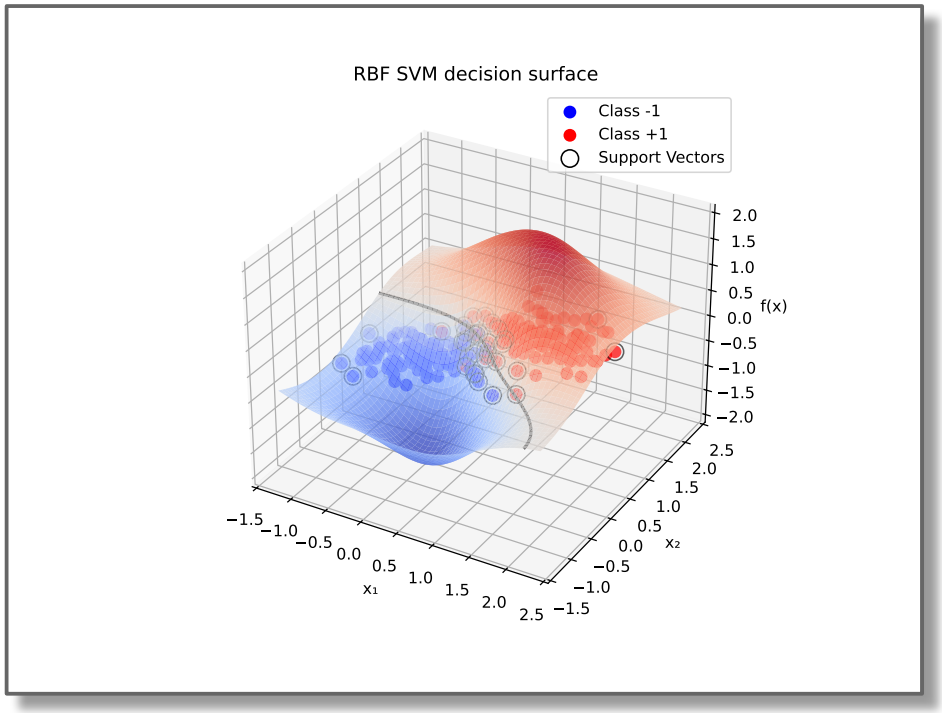



```
# Support Vector Machine example
from sklearn import svm
from sklearn.metrics import accuracy_score

clf = svm.SVC(kernel='linear', C=1.0)
clf.fit(X, y)

# 🔮 Predict on the test set
y_pred = clf.predict(X_test)

# 🧮 Evaluate
acc = accuracy_score(y_test, y_pred)

→ acc = 97.5%
```

Support Vector Machine

- in the code we should use the *‘rbf’* kernel *(Radial Basis Function)*





END OF LECTURE 10